Name:- Priyam Das
Roll No-:- 2401MC04

OS Lab Assignment 9: Page Tables, Page Faults and Page Replacement

I worked on a fresh copy of xv6-public (x86). In the Makefile I removed -Werror and set -smp N,sockets=N. Two system calls were added: `vmprint` (number 22) and `getfaults` (number 23), and each kernel edit carries an `// A9 Qn` comment. To test, boot with `make qemu-nox CPUS=1` and run `pttest`, `faulttest` and `pagerepl`. The logs are in output_logs/ and the screenshots are in screenshots/ (also appended at the end of this report).

Q1: Printing the page table (vmprint)

What I changed. `vmprint(pgdir)` in vm.c is modelled on `freevm()`. The outer loop goes through the page-directory entries from 0 up to `PDX(KERNBASE)`, which leaves out the kernel half, and every present entry is counted as one page-table page. For each such entry the inner loop goes through the table (`P2V(PTE_ADDR(pde))`) and prints each present PTE with its virtual address (`PGADDR(i,j,0)`), physical address (`PTE_ADDR(pte)`) and the P/W/U flags. Only pages with PTE_U set are added to the user-page total, and the page-table count includes the page directory (+1). The system call is hooked up in syscall.h, syscall.c, sysproc.c, user.h, usys.S and defs.h.

What the output shows. At start-up the process has three PTEs: code/data at 0x0 (PWU), the guard page at 0x1000 (PW-, because exec clears U) and the stack at 0x2000. So there are 2 user pages and 2 page-table pages. After `sbrk(4*4096)`, `pttest` writes to all four new pages (necessary because of the lazy allocation in Q2), and exactly four new PWU entries show up at 0x3000-0x6000, bringing the total to 6 user pages. Their physical frames are not contiguous. The page-table count remains 2 because everything still lies in the first 4 MB.

Q2: Lazy allocation and page faults

What I changed.

- `sys_sbrk`: for n > 0 it just adds n to `sz`, after rejecting an overflow or a result at or above KERNBASE. Negative n still goes through `growproc`.
- `copyuvm`: skips (continue) a missing page table or a non-present PTE instead of calling panic. `deallocuvm` already ignores non-present PTEs, so I only added a comment. `mappages` is no longer static.
- `trap.c`: the `T_PGFLT` case calls `lazyfault()`. It reads `rcr2()` and rejects protection faults (err & 1, such as the guard page) and any address at or above sz. Otherwise it runs `kalloc()`, `memset(0)`, `mappages(PGROUNDDOWN(va), PTE_W|PTE_U)` and `p->faults++`, then returns so the instruction restarts. An invalid user-mode fault prints the pid and address and sets killed. A kernel-mode fault inside a system call (for example `read()` into an untouched buffer) is handled the same way; only a kernel fault on a truly bad address still panics.
- `proc.h` / `proc.c`: new field `int faults`, set to zero in allocproc. `exec.c` clears it once the new image is committed. `getfaults()` returns `myproc()->faults`.

What the output shows. After `sbrk(32*4096)` the counter is still 0. The first write to each of pages 0, 7, 15, 23 and 31 adds one fault, giving 5, and rewriting pages that are already mapped adds nothing. A forked child that writes to sz + 4096 is killed (`pid 4 ... invalid page fault at addr 0x24000 (sz 0x23000)`), while the parent carries on. The stock `usertests` still ends with ALL TESTS PASSED.

A bug I hit. The first run printed 7 instead of 5. The shell's forked child calls malloc inside `parsecmd()` (2 lazy faults) before it execs, and that count was inherited by the new program. Resetting the counter in `exec()` fixed it.

Q3: FIFO vs LRU (pagerepl)

The program. Usage is `pagerepl [frames refs...]`; references may be comma-separated, since the xv6 shell accepts at most 10 arguments. With no arguments it uses the given reference string and 3 frames. `simulate()` keeps `frame[]` and `last[]` arrays. FIFO evicts the frame under a rotating pointer (the page loaded earliest). LRU refreshes `last[]` on every hit and evicts the smallest value; empty frames hold -1, so they get filled first. For each reference it prints the frames and hit/FAULT, and at the end a table for 3, 4 and 5 frames. Hit ratios use integer arithmetic because xv6 printf has no %f.

Frames	FIFO faults	FIFO hit ratio	LRU faults	LRU hit ratio
3	15	25.00%	12	40.00%
4	10	50.00%	8	60.00%
5	9	55.00%	7	65.00%

Observations. LRU does better because it holds on to page 0, which is reused often, while FIFO throws it out only because it is old. Six faults in every row are compulsory (there are 6 distinct pages). On the string 1,2,3,4,1,2,5,1,2,3,4,5, FIFO shows Belady's anomaly (9 faults with 3 frames, 10 with 4), whereas LRU, being a stack algorithm, cannot (10, then 8). Screenshots for all three questions follow.

Appendix: terminal screenshots

Q1: pttest

```
$ make qemu-nox CPUS=1
...
$ pttest
=== page table at start-up ===
VA          PA          FLAGS
0x0  0xdee8000  PWU
0x1000  0xdee6000  PW-
0x2000  0xdee5000  PWU
valid user pages: 2, page-table pages: 2

=== after sbrk(4 * 4096), new region 0x3000 - 0x7000 ===
VA          PA          FLAGS
0x0  0xdee8000  PWU
0x1000  0xdee6000  PW-
0x2000  0xdee5000  PWU
0x3000  0xdfbc000  PWU
0x4000  0xdf76000  PWU
0x5000  0xdfbf000  PWU
0x6000  0xdfc0000  PWU
valid user pages: 6, page-table pages: 2
```

Q2: faulttest

```
$ make qemu-nox CPUS=1
...
$ faulttest
faults at start: 0
sbrk(32 * 4096) -> 0x3000, faults after sbrk: 0
wrote page 0 (addr 0x3000), faults = 1
wrote page 7 (addr 0xA000), faults = 2
wrote page 15 (addr 0x12000), faults = 3
wrote page 23 (addr 0x1A000), faults = 4
wrote page 31 (addr 0x22000), faults = 5
rewrote 2 already-touched pages, faults = 5
page faults handled: 5 (expected 5)
child 4: writing above sz at 0x24000
pid 4 faulttest: invalid page fault at addr 0x24000 (sz 0x23000)--kill proc
parent 3: child was killed, parent still alive, faults = 5
```

Q2: usertests still pass

```
$ make qemu-nox CPUS=1
...
$ usertests
usertests starting
arg test passed
createdelete test
createdelete ok
linkunlink test
...
fork test OK
bigdir test
bigdir ok
uio test
pid 593 usertests: trap 13 err 0 on cpu 0 eip 0x35e3 addr 0x24d1d004--kill proc
uio test done
exec test
ALL TESTS PASSED
```

Q3: pagerepl (FIFO/LRU trace and table)

```
$ make qemu-nox CPUS=1
...
$ pagerepl
reference string (20 refs): 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

--- FIFO, 3 frames ---
ref 7 :  7  -  -   FAULT
ref 0 :  7  0  -   FAULT
ref 1 :  7  0  1   FAULT
ref 2 :  2  0  1   FAULT
ref 0 :  2  0  1   hit
ref 3 :  2  3  1   FAULT
ref 0 :  2  3  0   FAULT
ref 4 :  4  3  0   FAULT
ref 2 :  4  2  0   FAULT
ref 3 :  4  2  3   FAULT
ref 0 :  0  2  3   FAULT
ref 3 :  0  2  3   hit
ref 2 :  0  2  3   hit
ref 1 :  0  1  3   FAULT
ref 2 :  0  1  2   FAULT
ref 0 :  0  1  2   hit
ref 1 :  0  1  2   hit
ref 7 :  7  1  2   FAULT
ref 0 :  7  0  2   FAULT
ref 1 :  7  0  1   FAULT
FIFO total page faults: 15

--- LRU, 3 frames ---
ref 7 :  7  -  -   FAULT
ref 0 :  7  0  -   FAULT
ref 1 :  7  0  1   FAULT
ref 2 :  2  0  1   FAULT
ref 0 :  2  0  1   hit
ref 3 :  2  0  3   FAULT
ref 0 :  2  0  3   hit
ref 4 :  4  0  3   FAULT
ref 2 :  4  0  2   FAULT
ref 3 :  4  3  2   FAULT
ref 0 :  0  3  2   FAULT
ref 3 :  0  3  2   hit
ref 2 :  0  3  2   hit
ref 1 :  1  3  2   FAULT
ref 2 :  1  3  2   hit
ref 0 :  1  0  2   FAULT
ref 1 :  1  0  2   hit
ref 7 :  1  0  7   FAULT
ref 0 :  1  0  7   hit
ref 1 :  1  0  7   hit
LRU total page faults: 12

frames | FIFO faults | FIFO hit ratio | LRU faults | LRU hit ratio
-------+-------------+----------------+------------+--------------
   3   |     15      |     25.00%     |     12     |    40.00%
   4   |     10      |     50.00%     |      8     |    60.00%
   5   |      9      |     55.00%     |      7     |    65.00%
```

Q3: Belady's anomaly

```
$ make qemu-nox CPUS=1
...
$ pagerepl 3 1,2,3,4,1,2,5,1,2,3,4,5
reference string (12 refs): 1 2 3 4 1 2 5 1 2 3 4 5

--- FIFO, 3 frames ---
ref 1 :  1  -  -   FAULT
ref 2 :  1  2  -   FAULT
ref 3 :  1  2  3   FAULT
ref 4 :  4  2  3   FAULT
ref 1 :  4  1  3   FAULT
ref 2 :  4  1  2   FAULT
ref 5 :  5  1  2   FAULT
ref 1 :  5  1  2   hit
ref 2 :  5  1  2   hit
ref 3 :  5  3  2   FAULT
ref 4 :  5  3  4   FAULT
ref 5 :  5  3  4   hit
FIFO total page faults: 9

--- LRU, 3 frames ---
ref 1 :  1  -  -   FAULT
ref 2 :  1  2  -   FAULT
ref 3 :  1  2  3   FAULT
ref 4 :  4  2  3   FAULT
ref 1 :  4  1  3   FAULT
ref 2 :  4  1  2   FAULT
ref 5 :  5  1  2   FAULT
ref 1 :  5  1  2   hit
ref 2 :  5  1  2   hit
ref 3 :  3  1  2   FAULT
ref 4 :  3  4  2   FAULT
ref 5 :  3  4  5   FAULT
LRU total page faults: 10

frames | FIFO faults | FIFO hit ratio | LRU faults | LRU hit ratio
-------+-------------+----------------+------------+--------------
   3   |      9      |     25.00%     |     10     |    16.66%
   4   |     10      |     16.66%     |      8     |    33.33%
   5   |      5      |     58.33%     |      5     |    58.33%
```